- Some rational decimal numbers (like 0.1 in Base-10) cannot be represented precisely with the binary floating-point system (in Base-2)
- Comparison sometimes yields unexpected results, as do functions like `floor()` and `ceil()`

What can we do about it? Well, there's always rounding; that's pretty intuitive. Straight away, though, we run into problems with that. PHP supports four different rounding strategies with the built-in function `round()`:

```
var_dump(round(1.55, 1, PHP_ROUND_HALF_UP)); // If omitted, third argument
                                             defaults to this
var_dump(round(1.55, 1, PHP_ROUND_HALF_DOWN));
var_dump(round(1.55, 1, PHP_ROUND_HALF_EVEN));
var_dump(round(1.55, 1, PHP_ROUND_HALF_ODD));
```

```
Output:
float(1.6)
float(1.5)
float(1.6)
float(1.5)
```

In addition to needing to manage rounding strategies across your code, there's also some juggling to do with the second parameter of `round()`, which is the required precision (the number of decimal digits to round to). If you're handling very small numbers, the required precision is correspondingly large:

```
var_dump(round(1.51E-10, 10));
var_dump(round(1.51E-10, 11));
var_dump(round(1.51E-100, 100));
var_dump(round(1.51E-100, 101));
```

```
Output:
float(2.0E-10)
float(1.5E-10)
float(2.0E-100)
float(1.5E-100)
```